Sync Engine & Conflict Resolution

Architectural Design: Syncing Offline Edits

The sync engine is the layer that bridges the gap between a user's local device state and the authoritative server state. Its primary challenge is that clients are not always connected; a user may edit files on a plane, commit dozens of changes, reconnect, and expect everything to reconcile correctly with edits made by other devices in the interim. The protocol must handle this gracefully without data loss, duplication, or silent corruption.

The core of the sync architecture is a **local operation log** (oplog) maintained on each client. Every file mutation: create, edit, rename, delete is appended to this log with a sequence number and a causal timestamp before any network communication is attempted. When the client is offline, mutations queue in the oplog. When connectivity is restored, the client replays the log against the server in order, and the server processes each operation idempotently. This append-only local log is what makes offline-first sync tractable: the client never loses work because the source of truth for *local* state is always durable on disk.

The sync protocol itself operates in two phases. In the **push phase**, the client uploads any locally committed operations not yet acknowledged by the server, attaching file chunk references and a vector clock (discussed below) to each operation. In the **pull phase**, the client fetches any server-side operations it has not yet seen, identified by a monotonically increasing server sequence number called a **sync cursor**. The client stores its last-known cursor and uses it as a lower bound on each pull request, so it only fetches deltas rather than full state.

Distinguishing between time models is essential here. **Physical time** (wall-clock timestamps from gettimeofday or NTP) is unreliable across distributed nodes due to clock drift, jumps, and can go backward. Using physical timestamps to order operations leads to correctness bugs: an edit made at 10:00:01 on a device with a fast clock could appear to precede an edit made at 10:00:02 on a device with a slow clock, when causally the opposite is true. **Logical time**, specifically **Lamport clocks** or **vector clocks**, captures the *happened-before* relationship

between events without relying on synchronized wall clocks. A Lamport clock is a single integer incremented on every local event and updated on every message received (to ***max(local, received) + 1***). A vector clock extends this to a per-node integer array, giving full causal visibility: node A's vector [3, 1, 0] tells you that A has seen 3 of its own events, 1 from B, and 0 from C. **Causal ordering** is what the sync engine actually needs, not a total global order, but enough information to determine whether two edits are causally independent (and thus potentially conflicting) or whether one definitively happened after the other.

In practice, the sync engine uses vector clocks to annotate every operation. When the server receives two operations with non-comparable vector clocks, neither of which happened before the other, it knows they are concurrent and routes them to the conflict resolution service.

Clocks & Conflict: Reasoning About Concurrent Offline Edits

Perfect clock synchronization across distributed systems is not achievable in practice. NTP synchronizes clocks to within a few milliseconds under good network conditions, but under packet loss, asymmetric routing, or VM clock skew, errors can grow to hundreds of milliseconds or more. Google's TrueTime, used in Spanner, bounds clock uncertainty to single-digit milliseconds using GPS and atomic clocks, an extraordinary engineering investment unavailable to most systems. For a file sync system running on commodity hardware across heterogeneous client devices (laptops, phones, embedded clients), wall-clock ordering is simply not a correctness primitive. A client device's clock can be years off if a user manually sets it wrong. Depending on physical timestamps for causal ordering would mean a user with a misconfigured clock could silently overwrite a collaborator's recent edits with stale ones.

Consider a concrete scenario. User A and User B share a folder. Both go offline. A edits **report.docx** at local wall time 14:00. B edits the same file at local wall time 13:55, but B's clock is five minutes behind. Both reconnect simultaneously. If the server uses wall-clock timestamps to resolve the conflict, B's edit wins, even though A's edit may have been the most recent in real time. The data loss is silent: the server has no way to know whose clock was wrong.

Vector clocks resolve this correctly. When A and B both go offline with an identical base vector clock [A:5, B:3], then A's offline edit produces [A:6, B:3] and B's offline edit produces [A:5, B:4]. Neither vector dominates the other — they are concurrent by definition. The server detects this non-dominance and flags the pair as a conflict requiring resolution. No data is silently discarded; both versions are preserved and presented to the conflict resolution layer.

The conflict resolution policy itself is application-specific. For plain-text files, a three-way merge using the common ancestor version (the last version both clocks agree on) works well. For binary files (images, compiled artifacts), automatic merging is not meaningful; the system must either pick a winner (last-write-wins with a logged audit trail), fork the file into two conflict copies (Dropbox's model, creating a ***report (John's conflicted copy).docx***), or present a merge UI to the user. For structured documents (spreadsheets, JSON configs), operational transformation (OT) or conflict-free replicated data types (CRDTs) can allow automatic merging at the field or cell level, since individual operations can be commuted and reordered without changing the final state.

Stale reads are a related problem. When a client reconnects after a long offline period and issues a read before its pull phase completes, it may serve locally cached data that has been superseded on the server. The sync engine must distinguish between **read-your-own-writes consistency** (guaranteed within a single session by the local oplog) and **cross-device consistency** (not guaranteed until the pull phase catches up). Until the client's sync cursor matches the server's latest sequence number, reads should either be marked as potentially stale in the UI or deferred to the server. Surfacing this distinction honestly in the product, where a "syncing…" indicator, for instance, is preferable to silently returning stale data.

Idempotency & Retries: Safe Retry Patterns for the Sync Engine

Network failures during sync are not exceptional. A mobile client uploading a large file across a flaky LTE connection will experience partial uploads, TCP resets, and server-side timeouts

regularly. The sync engine must be designed so that retrying any operation after a failure produces exactly the same result as if the operation had succeeded on the first attempt. This property is **idempotency**, and it must be a first-class design constraint, not an afterthought.

The fundamental mechanism for idempotent uploads is a **client-generated operation ID** (also called an idempotency key). Before the client sends any request, it generates a stable, deterministic identifier for that operation, typically a UUID or a hash of the operation's content and context, and includes it in every retry of the same request. The server maintains a **deduplication log** (a short-lived table keyed on operation ID, typically with a TTL of 24–72 hours) that records the result of each operation it has already processed. If the server receives a request whose operation ID is already in the dedup log, it returns the cached result immediately without re-executing the operation. This converts a potentially destructive retry into a safe no-op.

For the **Upload Chunk** operation specifically, idempotency is achieved through **content-addressed storage**. Each file is split into fixed-size chunks (typically 4–8 MB), and each chunk is identified by the cryptographic hash of its contents (e.g., SHA-256). The chunk address *is* the chunk identity. If the client uploads the same chunk twice, whether due to a retry, a duplicate file, or a concurrent upload from another device, the storage layer detects the hash collision and deduplicates: it stores the chunk once, increments a reference count, and returns success to both callers. This means "Upload Chunk" is idempotent by construction: uploading a chunk that already exists is always safe and produces no side effects beyond confirming the chunk is present.

The client should also implement **resumable uploads** for large files. Before beginning an upload, the client queries the server for which chunks of the file's manifest are already present (using their hashes). The server responds with a bitmask or a list of missing chunk hashes. The client uploads only the missing chunks. If connectivity drops mid-upload and the client retries, it re-queries and uploads only what remains. This pattern, sometimes called a **chunked multipart upload with pre-flight check**, means large file uploads are restartable at chunk granularity rather than restarting from byte zero.

For **metadata operations** (committing a new file version, updating permissions, renaming a path), idempotency requires more care because these operations mutate shared state. The pattern

here is **compare-and-swap (CAS) with a version token**. The client reads the current metadata version token (an opaque integer or ETag), performs its mutation, and submits the result along with the token it read. The server accepts the write only if the token still matches, meaning no concurrent writer has intervened. If the token has changed, the server rejects the write with a conflict error, and the client must re-fetch the current state, re-apply its mutation, and retry with the new token. This is optimistic concurrency control: it assumes conflicts are rare, avoids locking, and handles conflicts explicitly when they occur.

Retry backoff policy is also critical. A client that retries failed requests immediately and aggressively during a server overload event will worsen the outage. The sync engine should implement **exponential backoff with jitter**: starting from a base delay (e.g., 500ms), each retry doubles the wait time up to a maximum (e.g., 60s), with a random jitter of ±20% added to prevent synchronized retry storms from thousands of clients hitting the server simultaneously at the same interval. Operations that receive a *429 Too Many Requests* or *503 Service Unavailable* response should respect any *Retry-After* header from the server, which signals a specific minimum wait time before the next attempt.

Together, content-addressed chunk deduplication, operation-ID-based server-side dedup, CAS for metadata, and backoff-with-jitter retry, these patterns make the sync engine robust to the full range of partial failures that occur in a distributed system operating at petabyte scale across millions of intermittently connected devices.